



Desafio Técnico – Estagiário Full Stack

Empresa: *ShoeTrack Industries*



1. Contexto do Problema

A **ShoeTrack Industries** é uma empresa de grande porte que produz milhares de pares de sapatos por dia.

Atualmente, o controle de produção é feito em planilhas, o que causa:

- Falta de rastreabilidade da produção
- Dificuldade de identificar gargalos
- Erros na contagem de estoque
- Falta de visibilidade por modelo e tamanho

Você deverá construir um **Sistema Web de Controle de Produção de Calçados**.



2. Objetivo do Sistema

Criar uma aplicação que permita:

- Cadastrar modelos de sapatos
 - Registrar ordens de produção
 - Atualizar status da produção
 - Controlar quantidade produzida
 - Visualizar relatórios simples
-

Stack Obrigatória

Frontend

- React (Vite ou CRA)
- TypeScript (obrigatório)
- Axios
- React Router
- CSS Modules ou Styled Components

Backend

- Node.js
- Express
- TypeScript
- Prisma OU Sequelize
- JWT para autenticação

Banco de Dados

- PostgreSQL
-



3. Funcionalidades Obrigatórias



3.1 Autenticação

- Login com email e senha
- JWT
- Rotas protegidas
- Middleware de autenticação

Perfis:

- Admin
 - Operador
-



3.2 Gestão de Modelos de Sapato

CRUD completo:

Campos:

- id
 - name
 - category (ex: Esportivo, Social, Infantil)
 - base_cost
 - created_at
-



3.3 Ordem de Produção

Campos:

- id
- model_id (FK)
- size (ex: 34-44)
- quantity_planned
- quantity_produced
- status (PLANNED | IN_PROGRESS | COMPLETED)
- start_date
- end_date

Funcionalidades:

- Criar ordem
 - Atualizar status
 - Atualizar quantidade produzida
 - Listar ordens
 - Filtrar por status
-



3.4 Dashboard Simples

- Total produzido hoje
 - Total em produção
 - Total concluído
 - Produção por modelo (gráfico simples opcional)
-

4. Modelagem do Banco de Dados (Esperada)

```
users
-----
id (uuid)
name
email (unique)
password
role (ADMIN | OPERATOR)
created_at
```

```
shoe_models
-----
id (uuid)
name
category
base_cost
created_at
```

```
production_orders
-----
id (uuid)
model_id (fk)
size
quantity_planned
quantity_produced
status
start_date
end_date
created_at
```



5. Requisitos Técnicos – Backend



Estrutura Esperada

```
src/
├─ controllers/
├─ services/
├─ routes/
├─ middlewares/
├─ database/
```

```
└─ utils/  
└─ app.ts
```

✓ Padrões Obrigatórios

- Separação Controller / Service
 - Validação com Zod ou Joi
 - Tratamento global de erros
 - Uso correto de HTTP status codes
 - Variáveis em `.env`
 - Senhas com bcrypt
 - Código organizado e legível
-

✗ Não Permitido

- SQL puro sem ORM
 - Código tudo dentro do controller
 - Senha salva sem hash
 - Hardcode de secrets
-



6. Requisitos Técnicos – Frontend

Estrutura Esperada

```
src/  
└─ pages/  
└─ components/  
└─ services/  
└─ hooks/  
└─ routes/  
└─ types/
```

Boas Práticas Esperadas

- Componentes reutilizáveis
- Separação de lógica e UI
- Uso correto de hooks

- Tratamento de loading e erro
 - Proteção de rotas
 - Tipagem forte (TypeScript)
-



7. Regras de Segurança

- JWT expira em 1h
 - Senha criptografada
 - Middleware protege rotas privadas
 - Não expor stack trace para usuário
-



8. Entrega Esperada

O candidato deve entregar:

- Repositório Git
 - README com:
 - Como rodar o projeto
 - Como rodar o banco
 - Variáveis de ambiente
 - Scripts:
 - `npm run dev`
 - `npm run build`
 - `npm run migrate`
-



9. Diferenciais (Pontuação Extra)

- Docker (backend + postgres)
- Testes (Jest)
- Testes E2E
- Logs estruturados
- Paginação

- Filtros avançados
- Deploy em Vercel/Render
- Swagger



10. Critérios de Avaliação

Critério	Peso
Organização do código	★★★★
Clareza da modelagem	★★★
Qualidade do frontend	★★★
Boas práticas	★★★★
Segurança	★★★
Extras	★★



11. O que esse desafio avalia

- Capacidade de modelar domínio
- Organização
- Noção de arquitetura
- Boas práticas
- Pensamento crítico
- Comunicação via README